Tema 5: Clases

Fundamentos de Programación

Francisco J. Rodríguez Díaz
(fjrodriguez@decsai.ugr.es)
Departamento de Ciencias de la Computación e I.A.

Universidad de Granada

Tabla de Contenidos I

1 Clases

Contenido del tema

1 Clases

Problema de las estructuras

No permiten *ocultar* o *proteger* los datos.

Ejemplo

```
1  struct CuentaBancaria {
2     string dni_dueño;
3     float saldo;
4  };
5
6  void transpaso(CuentaBancaria &origen, CuentaBancaria &destino, float cantidad) {
7     if (origen.saldo < cantidad) {
8        // error ...
9     }
10    else {
11       origen.saldo -= cantidad;
12       destino.saldo += cantidad;
13    }
14
15 }
```

Problema de las estructuras

Bien

La forma normal de modificar las cantidades es:

```
transferencia(cuenta1, cuenta2, cantidad);
```

No se garantiza

Pero no se impide hacer directamente.

```
// Le robamos al banco
cuenta2.saldo += 1e6;
```

Problema de las estructuras

Bien

La forma normal de modificar las cantidades es:

```
transferencia(cuenta1, cuenta2, cantidad);
```

No se garantiza

Pero no se impide hacer directamente.

```
// Le robamos al banco
cuenta2.saldo += 1e6;
```

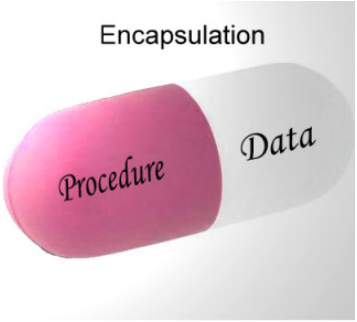
Encapsulación

- Se debería de impedir acceder directamente al atributo **saldo**, y sólo usar la función transferencia.

Encapsulación

Encapsulación

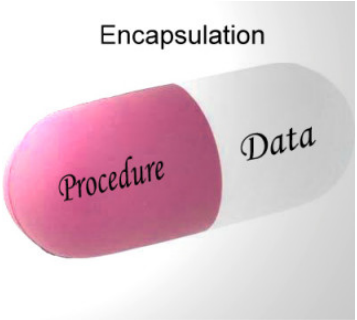
- Implica aunar datos y comportamientos en un único módulo.
- No se accede a los campos directamente, sólo por medio de un interfaz (función/método).



Encapsulación

Encapsulación

- Implica aunar datos y comportamientos en un único módulo.
- No se accede a los campos directamente, sólo por medio de un interfaz (función/método).



Ventajas de la encapsulación

- Garantiza un **acceso seguro** a los datos.
- Permite garantizar que los datos siempre son válidos.
- Se podría cambiar los campos de la estructura y el uso de la clase sería igual, si no se cambia el API (funciones de acceso).

Estructuras y Clases

C sólo tiene estructuras

- Permiten crear nuevos tipos compuestos: Punto, Alumno, ...
- Permite tratar de forma más cómoda.
- No encapsula.

C++ posee clases

- Son estructuras con funciones asociadas.
- No se accede a los atributos, si no a las funciones que los usan.
- Al acceder mediante métodos se *fomenta* la **encapsulación**.

Ejemplo de Clases

Ejemplo contador

```
5  class Contador {
6    int contador;
7  public:
8    int valor() {
9      contador += 1;
10     return contador;
11   }
12
13   void inicia() {
14     contador = 0;
15   }
16 };
```

```
18 int main(void)
19 {
20   Contador cont;
21
22   cont.inicia();
23   cout <<"Contador: ";
24   cout <<cont.valor() <<endl;
25   cout <<"Contador: ";
26   cout <<cont.valor() <<endl;
27   return 0;
28 }
```

Ejemplo de Clases

```
5  class CuentaBancaria {
6    float saldo;
7
8  public:
9    void transfiere(CuentaBancaria &destino, float cantidad) {
10     if (saldo >= cantidad) {
11       saldo -= cantidad;
12       destino.saldo += cantidad;
13     }
14   }
15
16   CuentaBancaria(float micantidad) {
17     saldo = micantidad;
18   }
19
20   float obtenerSaldo() {
21     return saldo;
22   }
23 };
```

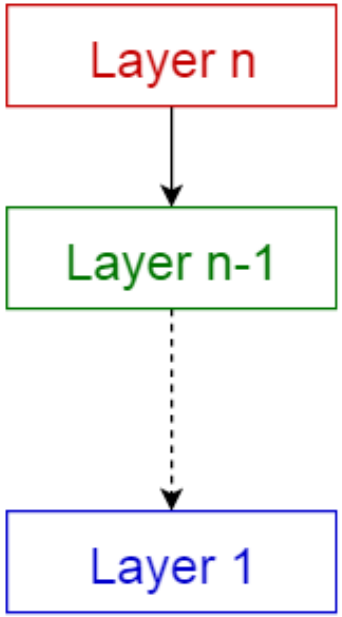
Ejemplo de Clases

```
25 int main(void)
26 {
27   CuentaBancaria cuenta1 (1000);
28   CuentaBancaria cuenta2 (0);
29
30   cout <<"Antes: " <<endl;
31   cout <<"C1 tiene " <<cuenta1.obtenerSaldo() <<endl;
32   cout <<"C2 tiene " <<cuenta2.obtenerSaldo() <<endl;
33
34   cuenta1.transfiere(cuenta2, 500);
35
36   cout <<"Después: " <<endl;
37   cout <<"C1 tiene " <<cuenta1.obtenerSaldo() <<endl;
38   cout <<"C2 tiene " <<cuenta2.obtenerSaldo() <<endl;
39
40   // No compila
41   // cuenta2.saldo += 6000;
42   return 0;
43 }
```

Tipos de Programación: Procedimental/Procedural

Programación procedural

- Todo el código se divide en módulos (funciones).
- Se diseña haciendo funciones de *alto nivel* que utilice funciones de bajo nivel.

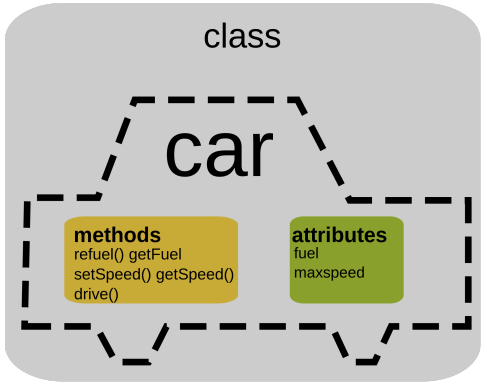


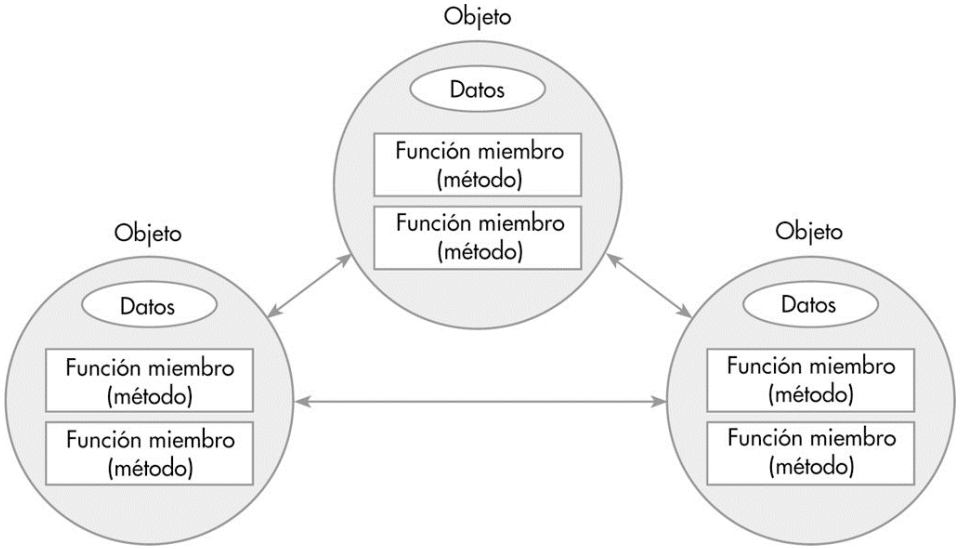

```
accede_BD() {
  ...
    }

lee_usuarios() {
  ...
    accede_BD();
  ...
    }

main() {
  ...
    usuarios = lee_usuarios();
```

Tipos de Programación: Orientada a Objetos

Programación orientada a objetos

- Todo el código se agrupa en **clases**.
- Una **clase** engloba unos datos, y unas funciones (llamadas métodos).
- Esos datos son compartidos entre esas funciones, no desde el resto del programa.





Tipos de Programación: Orientada a Objetos

Terminología

- Clase: Tipo de dato que definimos nosotros que contiene otros tipos de datos y funciones (métodos).
- Atributo o dato miembro: Dato de un tipo definido en una clase.
- Método: Función definida en una clase, puede usar sus parámetros y los atributos.
- Objeto: Variable del tipo de dato clase.

Ámbito público y privado

Ámbito

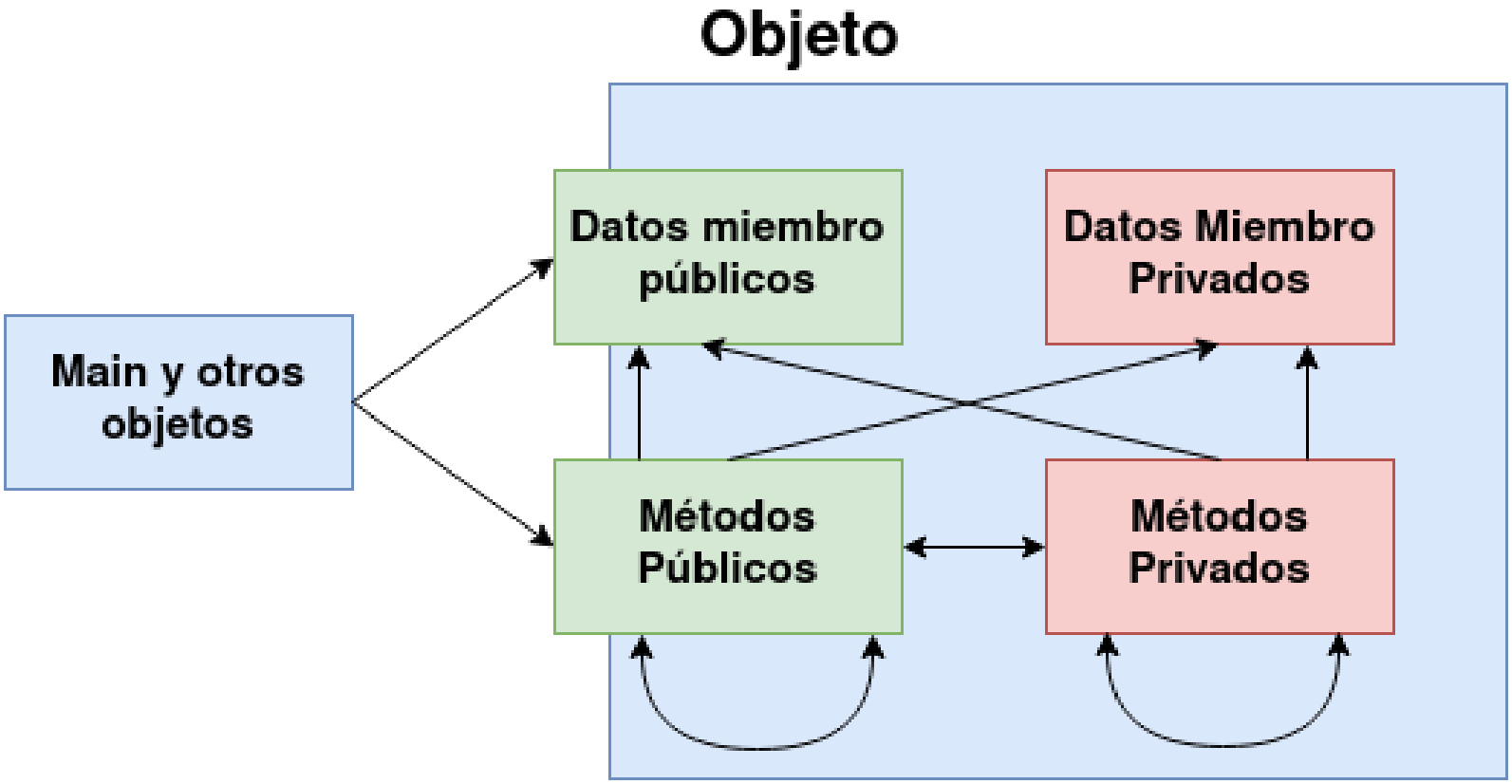
- Ámbito público (Public scope)
  - Se indica con el especificador de ámbito public
  - Los miembros públicos son visibles dentro y fuera del objeto.
- Ámbito privado (Private scope)
  - Se indica con el especificador de ámbito private (éste es el ámbito por defecto si no se pone nada)

Ámbito público y privado

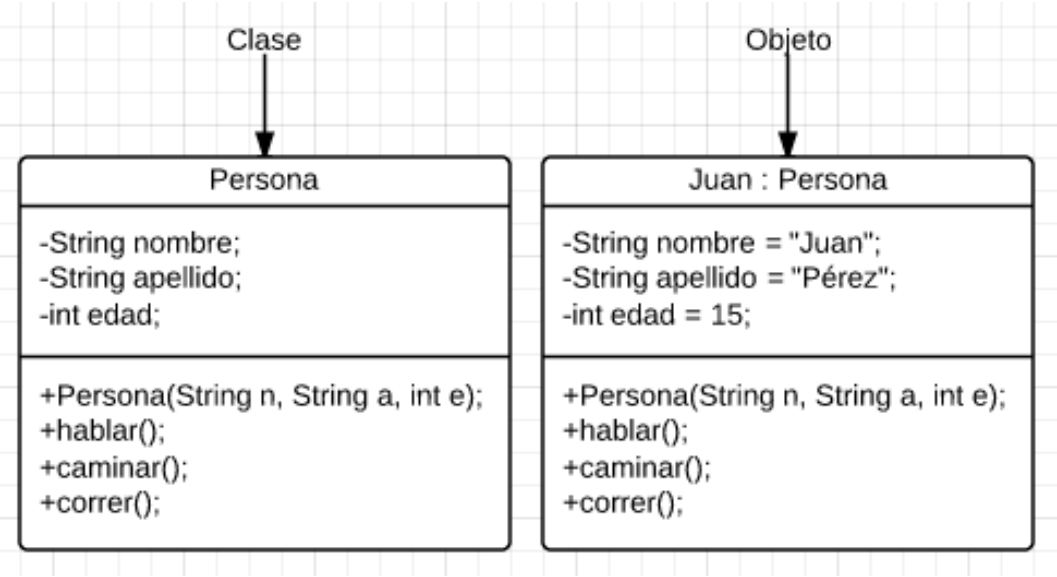
Ámbito

```
1  class Clase{
2  private:
3      int dato_privado;
4      void MetodoPrivado(){
5          ......
6      }
7
8  public:
9      int dato_publico;
10     void MetodoPublico(){
11         ......
12     }
13 };
14
15 int main(){
16     Clase objeto;
17     objeto.dato_publico = 4;
18     objeto.dato_privado = 4; // Error compilación
19     objeto.MetodoPrivado(); // Error compilación
20 }
```

Ámbito público y privado



Ejemplo de tipo de clase: Clase Persona



Ejemplo de tipo de clase

```
5  class Persona {
6    string nombre;
7    string apellido;
8    int edad;
9  public:
10   void set_nombre_completo(string minombre, string miapellido) {
11     nombre = minombre;
12     apellido = miapellido;
13   }
14
15   void set_edad(int miedad) {
16     edad = miedad;
17   }
18
19   void habla() {
20     cout <<"Me llamo " <<nombre <<" " <<apellido;
21     cout <<" y tengo " <<edad <<" años" <<endl;
22   }
23 };
```

Ejemplo de tipo de clase (Cont)

```
25 int main() {
26   Persona p1, p2;
27
28   p1.set_nombre_completo("Juan", "Pérez");
29   p1.set_edad(15);
30   p2.set_nombre_completo("Marisa", "Molina");
31   p2.set_edad(32);
32
33   p1.habla();
34   p2.habla();
35   return 0;
36 }
```

```
Me llamo Juan Pérez y tengo 15 años
Me llamo Marisa Molina y tengo 32 años
```

Sintaxis

Sintaxis

```
class <NuevoTipo> {
    <tipo1> <nombre1>;
    <tipo2> <nombre2>;
    ...

public:
    <metodo1>

    <metodo2>

    ...
};
```

Donde:

- **NuevoTipo** es el nombre del nuevo tipo.
- **tipo1**, **tipo2**, ... son los tipos de los atributos.
- Se puede acceder sólo a los métodos debajo de **public:**.
- Los métodos pueden acceder a los atributos.

Ejemplo de clase sencilla

```
5  class Position {
6    int x, y;
7  public:
8    void set(int mix, int miy) {
9      x = mix;
10     y = miy;
11   }
12
13   int get_x() {
14     return x;
15   }
16
17   int get_y() {
18     return y;
19   }
20
21   void mueve_izda() {
22     x -= 1;
23   }
```

Ejemplo de clase sencilla (Cont)

```
25    void mueve_dcha() {
26      x += 1;
27    }
28
29    void mueve_arriba() {
30      y -= 1;
31    }
32
33    void mueve_abajo() {
34      y += 1;
35    }
36
37  };
```

Sintaxis de uso

Creación

```
<NuevoTipo> variable;
```

```
41  Position punto;
```

Llamar a un método

```
variable.<metodo>(args);
```

Ejemplo de uso de métodos con parámetros:

```
43  punto.set(300, 300);
```

y sin parámetros:

```
51  cout <<"Nueva posición: " <<punto.get_x();
52  cout <<", " <<punto.get_y() <<endl;
```

Ejemplo de uso de Posición más completo

```
39 int main(void)
40 {
41   Position punto;
42
43   punto.set(300, 300);
44
45   for (int i = 0; i < 100; i++) {
46     punto.mueve_abajo();
47     punto.mueve_izda();
48     punto.mueve_izda();
49   }
50
51   cout <<"Nueva posición: " <<punto.get_x();
52   cout <<", " <<punto.get_y() <<endl;
53
54   return 0;
55 }
```

Constructor

Constructor

- A menudo interesa tener un método como **set** que inicia todos los atributos de la clase.
- **constructor** es un un método con **igual nombre que la clase** que inicia.
- El constructor debe ser llamado al iniciar la clase.

```
5  class Position {
6    int x, y;
7  public:
8    void set(int mix, int miy) {
9      x = mix;
10     y = miy;
11   }
12
13   int get_x() {
```

Constructor

Constructor

- A menudo interesa tener un método como **set** que inicia todos los atributos de la clase.
- **constructor** es un un método con **igual nombre que la clase** que inicia.
- El constructor debe ser llamado al iniciar la clase.

```
5  class Position {
6    int x, y;
7  public:
8    void set(int mix, int miy) {
9      x = mix;
10     y = miy;
11   }
12
13   int get_x() {
```

```
5  class Position {
6    int x, y;
7  public:
8    Position(int mix, int miy) {
9      x = mix;
10     y = miy;
11   }
12
13   int get_x() {
```

Uso del Constructor

Uso

- Se llama implícitamente al crear la variable.
- Si tiene parámetros debe de pasarse entre paréntesis.

```
41  Position punto;
42
43  punto.set(300, 300);
44
45  for (int i = 0; i < 100; i++) {
```

Uso del Constructor

Uso

- Se llama implícitamente al crear la variable.
- Si tiene parámetros debe de pasarse entre paréntesis.

```
41  Position punto;
42
43  punto.set(300, 300);
44
45  for (int i = 0; i < 100; i++) {
```

```
41  Position punto(300, 300);
42
43  for (int i = 0; i < 100; i++) {
```

Uso del Constructor

Lista inicialización

- A partir de C++11.
- Construcciones del tipo <dato_miembro>(<valor_inicial>) separadas por coma.
- Entre el segundo parentesis y la llave de inicio del constructor, comenzando por :

Ejemplo

```
class Position {
    int x, y;

public:

    Position(int mix, int miy): x(mix), y(miy) {
    }
```

Métodos Get y Set

Métodos Get y Set

- Permiten realizar comprobaciones de forma que los parámetros actuales sean correctos.
- De esta maner no se delega esta comprobación al usuario de la clase.

Ejemplos

- En una **CuentaBancaria** la cantidad a ingresar ha de ser positiva.
- En una **Fecha** los días deben estar entre 1 y 31 y además depende del mes (que tiene que estar entre 1 y 12).
- En una **Circunferencia** no permitimos radios negativos

Métodos Get y Set

Ejemplo

```
class CuentaBancaria{
    ......
    void Ingresa(double cantidad){
        if (cantidad > 0)
            saldo = saldo + cantidad;
    }
    void Retira(double cantidad) {
        if (saldo - cantidad > 0)
            saldo = saldo - cantidad;
    }
};

int main(){ // Un programa
    ......
    cuenta.Ingresa(ingreso);
    cuenta.Retira(retirada);
    ......
}
```

Cuenta Bancaria

Ejemplo

```
class CuentaBancaria{
private:
    double saldo;
    string identificador;

    string GetCodigoControl(string id_CCC){
        .......
    }
public:
    CuentaBancaria(string identificador_CCC): saldo(0.0){
        SetIdentificador(identificador_CCC);
    }

    void SetIdentificador(string identificador_CCC){
        string codigo_control;
        codigo_control = GetCodigoControl(identificador_CCC);
        identificador = "IBAN ES" + codigo_control +
            " " + identificador_CCC;
    }
};
```

Cuenta Bancaria

Ejemplo

```
1  int main(){
2      CuentaBancaria cuenta("20310381450100007510");
3      // cuenta: "IBAN ES 17 20310381450100007510", 0.0
```

Clase Secuencia Caracteres

SecuenciaCarcteres	
- const int	TAMANIO
- char	v[TAMANIO]
- int	util
+	SecuenciaCaracteres()
+ int	Utilizados()
+ int	Capacidad()
+ void	Aniade(char nuevo)
+ void	Modifica(int indice, char nuevo)
+ char	Elemento(int indice)

Clase Secuencia Caracteres

```
class SecuenciaCaracteres{
private:
   static const int TAMANIO = 10000;
   char v[TAMANIO];
   int util;

   bool EsCorrecta(int posicion){
      return 0 <= posicion && posicion < util;
   }

public:
   SecuenciaCaracteres()
      :util(0) {
   }

   int Utilizados(){
      return util;
   }

   int Capacidad(){
      return TAMANIO;
   }
```

Clase Secuencia Caracteres

```
1   public:
2      void Aniade(char nuevo){
3         if (util < TAMANIO){
4            v[util] = nuevo;
5            util++;
6         }
7      }
8
9      void Modifica(int posicion, char nuevo){
10        if (EsCorrecta(posicion))
11           v[posicion] = nuevo;
12     }
13
14     char Elemento(int indice){
15        return v[indice];
16     }
```

Clase Secuencia Caracteres

Más métodos

```
public:
    void Inserta(int pos_insercion, char nuevo){
      if (util < TAMANIO  &&  pos_insercion >= 0
         &&  pos_insercion <= util){

         for (int i = util ; i > pos_insercion ; i--)
            v[i] = v[i-1];

         v[pos_insercion] = nuevo;
         util++;
      }
   }

   void EliminaTodos(){
      util = 0;
   }
};
```

Clase Secuencia Caracteres

Main

```
1  int main(){
2      SecuenciaCaracteres secuencia;
3      int num_elementos;
4
5      secuencia.Aniade('h'); //h
6      secuencia.Aniade('o'); //ho
7      secuencia.Aniade('l'); //hol
8      secuencia.Aniade('a'); //hola
9
10     num_elementos = secuencia.Utilizados()
11
12     for (int i = 0; i < num_elementos; i++)
13         cout << secuencia.Elemento(i) << " ";
14
15     secuencia.v[9] = 'g' ; // Error compilac. Dato privado.
16     secuencia.Modifica(9, 'g'); //No lo modifica -> hola
17     secuencia.Modifica(0, 'l'); //lola
18 }
```

Clase SopaLetras

SopaLetras	
- const int	MAX_FIL
- const int	MAX_COL
- const int	util_col
- int	util_fil
+ char	m[MAX_FIL][MAX_COL]
+	SopaLetras(int num_columnas)
+	SopaLetras(SecuenciaCaracteres primera_fila)
+ int	CapacidadFilas()
+ int	FilUtilizadas()
+ int	ColUtilizadas()
+ char	Elemento(int fila, int columna)
+ SecuenciaCaracteres	Fila(int fila)
+ void	Aniade(SecuenciaCaracteres fila_nueva)
+ void	BuscaPalabra(SecuenciaCaracteres a_buscar, int &fil int &col)

Clase Sopa Letras

```
1  class SopaLetras{
2  private:
3     static const int MAX_FIL = 50;
4     static const int MAX_COL = 40;
5     char m[MAX_FIL][MAX_COL];
6     int util_fil;
7     const int util_col;
8  public:
9     SopaLetras(int num_columnas)
10       :util_fil(0), util_col(num_columnas)
11    {
12    }
13    SopaLetras(SecuenciaCaracteres primera_fila)
14       :util_fil(0), util_col(primera_fila.Utilizados())
15    {
16       Aniade(primera_fila);
17    }
18    int CapacidadFilas(){
19       return MAX_FIL;
20    }
21    int FilUtilizadas(){
22       return util_fil;
23    }
```

Clase Sopa Letras

```
int ColUtilizadas(){
   return util_col;
}
char Elemento(int fil, int col){
   return m[fil][col];
}
SecuenciaCaracteres Fila(int fil){
   SecuenciaCaracteres fila;

   for (int col = 0; col < util_col; col++)
      fila.Aniade(m[fil][col]);

   return fila;
}
string ToString(){
   string cad;

   for (int i = 0; i < util_fil; i++)
      cad = cad + "<" + Fila(i).ToString() + ">\n";

   return cad;
}
```

Clase Sopa Letras

```
1   void Aniade(SecuenciaCaracteres  fila_nueva){
2      int numero_columnas_nueva;
3
4      if (util_fil < MAX_FIL &&
5         fila_nueva.Utilizados() == util_col){
6
7         for (int col = 0; col < util_col ; col++)
8            m[util_fil][col] = fila_nueva.Elemento(col);
9
10        util_fil++;
11     }
12  }
```

Clase Sopa Letras

```
1     void BuscaPalabra (SecuenciaCaracteres a_buscar, int &fil, int &col){
2        bool encontrado, va_coincidiendo;
3        int  tamanio_a_buscar;
4
5        encontrado = false;
6        fil = col = -1;
7        tamanio_a_buscar = a_buscar.Utilizados();
8
9        if (tamanio_a_buscar <= util_col){
10          for (int f = 0; f < util_fil && !encontrado; f++){
11             int col_encontrado = Fila(f).PosContiene(a_buscar);
12             encontrado = col_encontrado != -1;
13
14             if (encontrado){
15                fil = f;
16                col = col_encontrado;
17             }
18
19          }
20       }
21    }
22 };
```

Copia de Objetos

Operador de Asignación

- C++ lo implementa por defecto.
- Utilizando el operador = con objetos de la misma clase.
- Se copian los atributos públicos y privados.

Ejemplo

```
1  int main(){
2      Position p1(1,2);
3      Position p2(3,4);
4      p2 = p1;
5      cout << "(" << p2.get_x() << ", " << p2.get_y() << ")" << endl
6      // (1,2)
```

Copia de Objetos

Constructor copia

- Se proporciona de oficio también.
- Recibe un objeto ya creado de la misma clase.

Ejemplo

```
1  int main(){
2      Position p1(1,2);
3      Position p2(p1);
4      cout << "(" << p2.get_x() << ", " << p2.get_y() << ")" << endl
5      // (1,2)
```

Vectores de Objetos

Vectores de objetos

- Es recomendable que se disponga de constructor sin parámetros de esa clase.
- Al declarar el vector se crean tantos objetos como tamaño del vector.

Ejemplo

```
1  class MiClase{
2      ......
3  };
4  int main(){
5      const int TAMANIO = 50;
6      MiClase vector_de_objetos[TAMANIO]; // <- Se crean 50 objetos
7  }
```

Clase Punto2D y Recta

Punto2D	
- double	abscisa
- double	ordenada
+	Punto2D(double abscisaPunto, double ordenadaPunto)
+ void	SetCoordenadas(double abscisaPunto, double ordenadaPunto)
+ double	Abscisa()
+ double	Ordenada()

Recta	
- double	A
- double	B
- double	C
+	Recta(double coef_x, double coef_y, double coef_indep)
+ void	SetCoeficientes(double coef_x, double coef_y, double coef_indep)
+ double	CoeficienteA()
+ double	CoeficienteB()
+ double	CoeficienteC()
+ double	Ordenada_en(double x)
+ double	Abscisa_en(double y)
+ string	ToString()
+ bool	Contiene(Punto2D punto)

Clase Punto2D y Recta

```
bool SonIguales(double uno, double otro) {
    return abs(uno-otro) <= 1e-6;
}

class Recta{
private:
    double A, B, C; // Ax + By + C = 0
public:
    ......
    bool Contiene(Punto2D punto){
        return SonIguales(A * punto.Abscisa() +
            B * punto.Ordenada() + C , 0.0);

    }
};
```

Clase Punto2D y Recta

```
1  int main(){
2      ......
3      contenido = una_recta.Contiene(un_punto);
4      if (contenido)
5          cout << "\nLa recta contiene el punto";
6      else
7          cout << "\nLa recta no contiene el punto";
```